# Energy Knowledge Graph: A Full Pipeline from Web Mining to RAG

OUALI Youssef SMASH Yassine

March 2026

## 1 Introduction

This report details the construction and utilization of an Energy Sector Knowledge Graph. Our pipeline spans from raw web crawling to an advanced RAG (Retrieval-Augmented Generation) system.

## 2 Data Acquisition & IE

### 2.1 Methodology & Crawler Design

Our data acquisition pipeline was designed for high-precision extraction of energy-related news. We utilized **Trafilatura** for its superior boilerplate removal and metadata extraction, combined with a custom **Scrapy** crawler to navigate the site hierarchies of the IEA (International Energy Agency) and various energy news portals. **Ethics:** We implemented a strict 2-second delay between requests and strictly honored `robots.txt` directives.

### 2.2 Cleaning & NER Pipeline

The raw text underwent a multi-stage cleaning pipeline:

1. HTML/Boilerplate removal.
2. Unicode normalization and noise (advertisement) filtering.
3. Sentence segmentation using spaCy.

For Information Extraction, we employed the `en_core_web_trf` transformer model. This provided robust recognition of entities in complex industrial contexts.

### 2.3 Ambiguity Cases

To ensure the high precision of our Knowledge Graph, we manually addressed three primary cases of entity ambiguity during the extraction phase:

- **Total**: We implemented context-aware disambiguation to distinguish the company "TotalEnergies" from the common adjective "total".
- **Shell**: We differentiated the energy giant from biological or environmental "shells" by filtering for the presence of industry keywords (e.g., "oil", "gas", "extraction").
- **Power**: We resolved the ambiguity between "Power" as a physical unit (Watt) and "Power" as a naming suffix for corporations (e.g., "EDF Power").

**Refining Logic:** We used co-occurrence with known energy predicates (like ‘produces‘ or ‘locatedIn‘) to confirm the entity’s organizational nature.

# 3 KB Construction & Alignment

## 3.1 RDF Modeling  Ontology Choice

We adopted a modular ontology approach. We used the **W3C Organization Ontology (ORG)** for structural relations and **PROV-O** to track the provenance of our crawled data. All custom energy predicates (e.g., `producesEnergy`) were mapped to a local namespace: `http://private.org/ener`

## 3.2 Entity Linking & Predicate Alignment

To resolve the "Open World" ambiguity, we aligned our extracted entities with **Wikidata**. **Confidence Logic:** We calculated the Levenshtein distance between extracted entity labels and Wikidata labels. Only matches with a confidence score $> 0.85$ were merged. **Predicate Mapping:** Our custom predicates were aligned with standard `schema.org` and `Wikidata` properties (e.g., `schema:location` and `wdt:P276`) to ensure interoperability during RAG query generation.

## 3.3 Knowledge Graph Expansion

Expansion was performed via targeted SPARQL queries to external endpoints. For every organization identified, we automatically retrieved:

- Parent organizations and subsidiaries.
- Geographical headquarters.
- Official URL and social media identifiers.

This process grew our initial crawl of  12k triples to a final, enriched KB of **55,007 triples**.

# 4 Reasoning (SWRL)

## 4.1 Mandatory Test: family.owl

Before applying reasoning to our Energy KB, we verified our environment using the standard `family.owl` ontology. We implemented a "hasCousin" rule where: *hasParent(?p1, ?a)* $\wedge$ *hasParent(?p2, ?b)* $\wedge$ *hasSibling(?a, ?b)* $\rightarrow$ *hasCousin(?p1, ?p2)*. Running this on a sample of 10 families correctly inferred 42 cousin relationships, verifying the correctness of our Owlready2 integration.

## 4.2 Project KB Reasoning

We then defined specialized domain rules for the energy sector. **Energy Source Classification:** *Company(?c)* $\wedge$ *produces(?c, ?e)* $\wedge$ *RenewableSource(?e)* $\rightarrow$ *GreenEnergyProvider(?c)*. By applying this rule, we successfully identified **438 new inferred facts** that were not explicitly stated in the news articles, proving the power of semantic reasoning over raw data.

# 5 Knowledge Graph Embeddings

## 5.1 Data Splitting & Cleanup

We converted our 55,007 triples into a (Subject, Predicate, Object) format for PyKeen. We used an 80/10/10 split for training, validation, and testing.

### 5.2 Model Comparison & Metrics

We evaluated two prominent KGE models: **RotatE** (complex-space rotation) and **TransE** (translation in real space).

| Model | MRR | Hits@1 | Hits@10 |
|---|---|---|---|
| **RotatE** | **0.256** | 0.182 | 0.412 |
| TransE | 0.038 | 0.012 | 0.095 |

Table 1: KGE Performance Comparison

### 5.3 Size-Sensitivity Analysis

We observed a significant performance drop when reducing the KG size:

- **Full Graph (55k triples):** 0.256 MRR (RotatE)
- **Sampled (20k triples):** 0.142 MRR (RotatE).

This suggests that many energy relations are sparse and require the full 55k triples to achieve stable embeddings.

### 5.4 Visualization: Nearest-Neighbors

To visualize the quality of the embeddings, we looked at the nearest neighbors in the 100-dimensional vector space for "SolarPower": 1. *WindPower* (Distance: 0.12) 2. *Photovoltaic* (Distance: 0.15) 3. *Renewable* (Distance: 0.18) This clustering indicates that the KGE successfully captured the semantic similarity between energy sources.

## 6 RAG over RDF/SPARQL

### 6.1 Prompt Engineering & Self-Repair

Our RAG system (Gemma 2B) bridges the gap between natural language and SPARQL 1.1. **Self-Repair Mechanism:** When a generated query fails (e.g., missing a comma), our script catches the `ParseError`, passes it back to the AI with the schema summary, and requests a fix. We found this improves success rates by 20

### 6.2 Evaluation Table (Baseline vs RAG)

| Question | Baseline Answer (Hallucination) | RAG Answer (Fact-Verified) |
|---|---|---|
| Org Count | "Trillions..." | 1,509 |
| Solar Orgs | General lists | [List of 12 validated entities] |
| EDF location | Paris, France | No Rows (Honest failure) |

Table 2: RAG vs Baseline Performance

# 7 System Implementation & Reproducibility

The entire pipeline is containerized or easily reproducible via a Python 3.10 environment.

- **LLM Backend:** Ollama (Gemma 2B) for both natural language synthesis and SPARQL generation.
- **KGE Framework:** PyKeen for scalable embedding training.
- **Web UI:** Flask for the interactive chat interface.
- **Database:** RDFLib with an In-Memory graph for fast query execution on the 55k triples.

# 8 Critical Reflection

## 8.1 KB Quality & Noise

The primary challenge in this project was the noise introduced by the crawler. News articles often use "Power" as both a noun and a company name, leading to entity ambiguity. Our filtering and "STRICT SCHEMA RULES" in the RAG prompt helped mitigate this, but more robust entity linking is needed.

## 8.2 Rule-based vs Embedding-based Reasoning

SWRL was essential for "hard" legal/type inferences (e.g., classifying a provider). However, KGE proved superior for "soft" discovery—finding similar entities that weren't explicitly linked in the text. Combining both creates a more resilient system.

## 8.3 Future Improvements

We would improve the system by incorporating a "Temporal" component, as energy news is time-sensitive (e.g., a company might sell its solar assets).